Simulation DEVS d'une Attaque par Injection SQL et Évaluation d'un Pare-feu Adaptatif

Astan Tangara

Master 2 Ingénierie Logicielle
Faculté des Sciences et Techniques de Bamako

Avril 2026

Plan de la présentation

1. Introduction
2. Formalisme DEVS
3. Modélisation du système de sécurité
4. Implémentation technique
5. Résultats expérimentaux
6. Discussion et Perspectives

Introduction

Contexte du projet

- Les applications web font face à des menaces constantes.
- **Injections SQL** : Dans le top 10 des risques de l'OWASP.
- **Problématique** : Comment évaluer l'efficacité des systèmes de défense de manière formelle ?
- **Approche** : Utilisation du formalisme DEVS pour la modélisation et la simulation.

Introduction

Objectifs du travail

1. **Modélisation formelle** : Création d'une chaîne Attacker $\rightarrow$ Firewall $\rightarrow$ Database.
2. **Évaluation comparative** : Tester 4 scénarios (baseline, attaque, défense statique, défense adaptative).
3. **Implémentation** : Développement en Java via le framework *SimStudio*.
4. **Analyse** : Mesurer l'efficacité par des métriques quantitatives.

Démarche suivie

- Analyse des concepts théoriques DEVS.
- Conception des modèles atomiques et couplés.
- Implémentation logicielle.
- Validation et simulation.
- Analyse des résultats.

Formalisme DEVS

Concepts fondamentaux

- **DEVS** : *Discrete Event System Specification*.
- Cadre mathématique pour les systèmes à événements discrets.
- Inventé par **Bernard Zeigler** en 1976.
- Séparation stricte entre le **modèle** (comportement) et le **simulateur** (exécution).

Formalisme DEVS

Le Modèle Atomique

Un modèle atomique DEVS est défini par le septuplet :

$$M = \langle X, Y, S, \delta_{ext}, \delta_{int}, \lambda, ta \rangle$$

X, Y : Événements d'entrée et de sortie.

S : Ensemble des états internes.

$\delta_{ext}, \delta_{int}$: Fonctions de transition externe et interne.

λ : Fonction de sortie.

ta : Fonction de durée de vie de l'état.

Formalisme DEVS

Le Modèle Couplé

Permet la composition de sous-modèles pour former un système complexe.

$$CM = \langle X, Y, D, \{M_d\}, EIC, EOC, IC, Select \rangle$$

- **D** : Noms des sous-modèles.
- **EIC, EOC, IC** : Couplages des ports (entrées/sorties/intermédiaires).
- **Select** : Gestion des conflits de simultanéité.

Modélisation

Architecture générale

Chaîne de traitement modélisée :

1. **Attacker** : Générateur de requêtes.
2. **Firewall** : Filtrage des requêtes (Statique ou Adaptatif).
3. **Database** : Traitement final et détection de compromission.

Modèles Atomiques

Modèle Attacker

- **Rôle** : Simule un pirate envoyant d'abord une requête saine, puis une attaque.
- **États** : 0 (prêt), 1 (requête normale), 2 (injection).
- **Sorties** : `NORMAL_QUERY`, `SQL_INJECTION_PAYLOAD`.
- **Timing** : $ta(s) = 1$ (un envoi par unité de temps).

Modèles Atomiques

Modèle Firewall (Statique)

- **Rôle** : Filtrage par signatures connues.
- **Entrées** : Requêtes entrantes.
- **Logique de sortie** (λ) :
  - Si `SQL_INJECTION_PAYLOAD` $\rightarrow$ `BLOCKED`.
  - Sinon $\rightarrow$ Transmission de la requête.
- **Délai** : $ta(s) = 0$ (réponse immédiate).

Modèles Atomiques

Modèle AdaptiveFirewall

- **Innovation** : Apprentissage adaptatif.
- **Mécanisme** : Si reçoit `COMPROMISED` de la DB, met à jour le flag `learned = true`.
- **Conséquence** : Bloque automatiquement les requêtes futures si l'attaque est apprise.

Modèles Atomiques

Modèle Database

- **Rôle** : Détection de la compromission.
- **Logique** (λ) :
  - Si `SQL_INJECTION_PAYLOAD` $\rightarrow$ `COMPROMISED`.
  - Sinon $\rightarrow$ `OK`.
- **Feedback** : Envoie une alerte au firewall adaptatif.

Modèle Couplé

Interconnexions (IC)

[Image de l'architecture de simulation SQL injection DEVS]

- **Attacker.output** → **Firewall.input**.
- **Firewall.output** → **Database.input**.
- **Database.output** → Sortie globale / Feedback Firewall.

Implémentation

Architecture logicielle

- **Package models** : Implémentation des modèles atomiques.
- **Package engine** : Moteur de simulation DEVS.
- **Interface** `AtomicModel` : Définit le contrat (*initialize*, *receive*, *output*, *internalTransition*) .

Implémentation

Exemple : Modèle Attacker

```
public class Attacker implements AtomicModel {
    private int step = 0;
    @Override
    public Object output() {
        if (step == 0) return "NORMAL_QUERY";
        if (step == 1) return "SQL_INJECTION_PAYLOAD";
        return null;
    }
    @Override
    public void internalTransition() {
        step++;
    }
}
```

Résultats

Configuration des tests

Quatre scénarios testés (avec répétitions chacun) :

1. **Baseline** : Sans attaquant, sans défense.
2. **Attack only** : Attaquant, pas de défense.
3. **Static defense** : Attaquant + firewall statique.
4. **Adaptive defense** : Attaquant + firewall adaptatif.

Résultats

Métriques de performance

Trois métriques clés collectées :

- **Blocked** : Requêtes malveillantes interceptées.
- **Compromised** : Nombre de compromissions réussies.
- **Forwarded** : Requêtes légitimes transmises.

Résultats

Synthèse des moyennes

Scénario	Blocked	Compromised	Forwarded
Baseline	0.0	0.0	0.0
Attack only	0.0	1.0	0.0
Static defense	1.0	0.0	1.0
Adaptive defense	1.0	0.0	1.0

Table – Moyennes des métriques par scénario

Analyse des résultats

- **Attack only** : 100% de compromission sans défense.
- **Static defense** : Protection parfaite contre les attaques connues grâce aux signatures.
- **Adaptive defense** : Résultat identique ici car le scénario est simple (une seule attaque connue).

Discussion

Apports de la méthode

- Efficacité de DEVS pour les systèmes complexes.
- Séparation nette entre logique métier et moteur d'exécution.
- Facilité de réutilisation des composants atomiques.

Discussion

Limites actuelles

- Trafic très simplifié (une seule attaque).
- Modèle purement déterministe.
- Absence de mesures de latence temporelle.

Perspectives

Travaux futurs

1. **Trafic réaliste** : Utiliser des distributions probabilistes.
2. **Apprentissage avancé** : Intégrer du Machine Learning dans le pare-feu.
3. **Métriques étendues** : Analyser les faux positifs et la latence.

Conclusion

Points clés

- Le formalisme DEVS est un outil robuste pour la cybersécurité.
- L'approche adaptative renforce la résilience face à l'inconnu.
- La simulation permet des analyses comparatives rigoureuses avant déploiement.

Merci pour votre attention

Questions ?